TASK 5

Goal-oriented coding agent

Course	Multi-Agent AI for Game Development
Author	David Monestel Aguilar
Game	Driftward Ace, an endless arcade dodger (Python, pygame-ce, runs in the browser with pygbag)
Repo and branch	github.com/david-mntl/driftward-ace/tree/tas-5-goal-oriented-coding-agent

1 In short

THE AGENT

A crew of Claude Code agents. Each agent is a Markdown file that sets its goal, what it reads, what it may write, and when it stops. I used this instead of the starter script; the brief allows any approach.

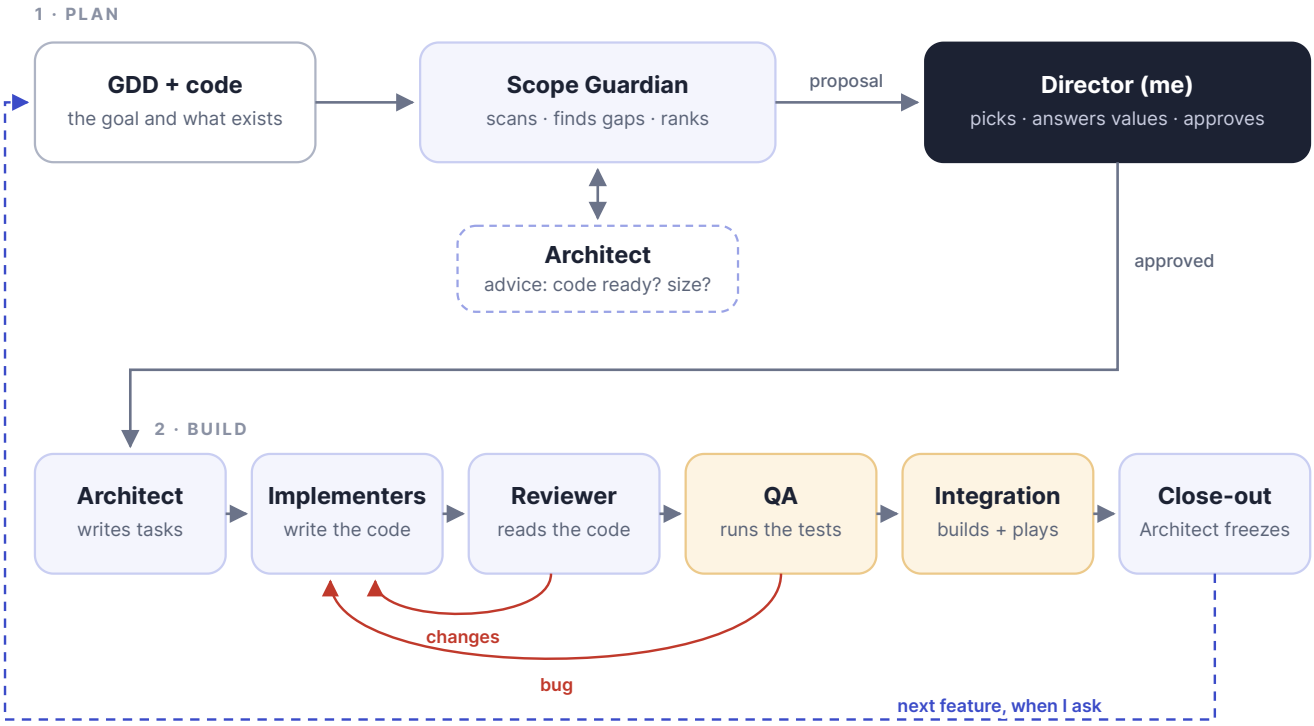
THE FEATURE

DRA-23, Lives, falls, game over and retry. It runs in the browser.

WHO DECIDES

The agents propose and build. I approve at fixed points. Project rule: *"The Director (the human) makes every final call"* (CLAUDE.md).

2 The loop



Each step writes a report file. The next agent starts only when the previous report says pass. That is the re-scan: after every write, the code is read and tested again. **Red**: fix loops. **Amber**: pass/fail gates.

3 What was asked vs. what we did

ASKED FOR	APPROACH USED	WHERE
Starter parts LLM loop, file scanner, GDD parser, print log	Claude Code runs the agents. Its file tools scan the repo. The GDD index tells each agent which GDD file to open. Each agent writes a report file instead of printing a log.	.claude/agents/, GDD index, docs/reports/
`priority_score()`	The **Scope Guardian** ranks candidates by the GDD's weekly schedule against what is already built. The earliest unfinished weekly goal comes first. The **Architect** (advice mode) checks the code to confirm what exists and what each candidate needs. Details in section 4.	**Schedule:** 13-scope-and-schedule.md **Ranking:** scope-guardian.md, "Asking the Architect" **Code check:** architect.md, mode 3 "Advice"
Gap detection	The **Scope Guardian** compares what the GDD requires with what already exists: the features already built (one scope report each) and the systems already designed (contracts). The **Architect** checks the game code. Values the feature needs but the GDD lacks are matched against the open-questions list.	**Required:** docs/design/gdd/ **Built:** docs/reports/scope/, docs/contracts/, game/ **Missing values:** 90-open-questions.md **Rules:** scope-guardian.md, "Read first, every run"
Code writing	The **Architect** splits the feature into task files. Each task file names one agent, and only that agent writes its code. DRA-23 had 5 tasks: game logic → Gameplay Implementer; hearts and Game Over box → UI Developer (2 tasks); tests → QA Engineer (2 tasks). The Integration Engineer then wired the feature into the main loop.	DRA-23 tasks architect.md, "Design run"
Re-scan after each write, stop when done	The `/implement` skill runs each task the Architect wrote. I start it with `/implement DRA-23`. If I approve ahead of time, the main Claude Code session runs it on its own as soon as the Architect finishes, all the way to close-out. After every write, the Reviewer reads the code and QA runs the tests again. A failure goes back to the agent that wrote the code. The loop stops when every task passes QA and the browser build passes. The Architect then closes the feature, and the agent waits for me before it proposes the next one.	/implement skill integration report

Design choice The loop runs one feature at a time and waits for me between features. Each cycle starts from the GDD, so the agent proposes from my documentation. The final call stays with me.

4 How the agent ranked the features

The agent did not use a numeric score. It checked each candidate against these GDD files:

GDD FILE	WHAT IT SAYS	HOW THE AGENT USED IT
`13-scope-and-schedule.md`	Week 1 goal: "Priority 1 objects. Lives. Restart." Done when the ship "can die".	Movement and scrolling were already built. The ship still could not die. So lives and restart were the open week-1 goal.
`01-vision.md`	Locked decision 2: a fall costs a life. You lose when "all lives lost". Three design pillars.	The code respawned the ship without taking a life. That broke a locked rule, and lives are needed before a run can be lost. Every candidate was checked against the pillars.
`05-lives-and-damage.md`	Locked values: 3 lives, 30 ticks of invulnerability, respawn at (320, 240).	The key values for lives already existed, so it could be built right away.
`07-objects.md`	The meteorite is the priority 1 object.	It was the main competitor.
`90-open-questions.md`	Lists every value still missing.	The meteorite's size, speed and warning (OBJ-01), Wave 0 (WV-05), the hazard warning (OB-01) and despawn (OB-03) were all missing. So the meteorite was blocked. A hit also means nothing without lives.

The ranking it returned

#	CANDIDATE	VERDICT	WHY
1	**A1, lives and falls**	PICKED	Week-1 goal, required by a locked rule, values ready.
2	A2, game over and retry	NEXT	Needs the game-over screen content (UI-04).
3	C, control card	NEEDS VALUES	Small, but needs its start key and text (FL-01, UI-03).
4	B, meteorite and Wave 0	BLOCKED	Blocked by missing values.

5 Example run

MY PROMPT

"I want you to propose to me the most suitable feature to develop now."

THE AGENT'S ANSWER (SHORTENED)

The Scope Guardian recommends **A1, "Lives and falls"**. [...] The ship can fall and respawn, but nothing costs a life, so the run can't be lost. The GDD's lose condition is "all lives lost", and week 1 of the schedule ends on "can die". [...] The Architect says A1 can be built on today's code at medium size.

GDD values A1 needs that are missing: blink speed (LD-04), lives as icons or a number (UI-02) [...]

Questions for you: Is A1 the feature to scope? [...]

It ended with questions for me instead of guessing the missing values. I picked A1, built A2 (game over and retry) in the same feature, and answered the open questions. My answers were written back into the GDD.

Full record: task-5-agent-proposal-conversation.md (the agent's full answer), scope report DRA-23.

6 Deliverables

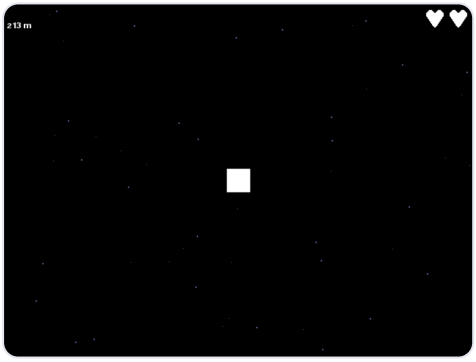
Source code

The agent definitions are the source code:

FILE	ROLE IN THE LOOP
`scope-guardian.md`	Reads the GDD, finds gaps, ranks and proposes features
`architect.md`	Gives advice, then splits the feature into tasks
`gameplay-implementer.md`, `ui-developer.md`	Write the code
`implementation-reviewer.md`, `qa-engineer.md`	Re-check the code and run the tests
`integration-engineer.md`	Builds for the browser and plays it
`/implement skill`	Runs the build loop from the first task to integration

Working feature: DRA-23

- **What the player sees:** 3 white hearts in the top-right corner. Falling off the bottom costs a heart. The ship respawns in the center, blinks, and waits for an arrow key. After 3 falls a Game Over box appears, and an arrow key starts a new run.
- **Code the agents wrote:** `core/run.py`, `ui/hud.py`, `ui/game_over.py`, `app/frame.py`, `tests`.
- **Proof it runs:** it runs in the browser. Start the game with F5 (see the README) and play it. The integration report also passed every browser check. All 5 QA reports passed.



A fall cost a heart. The ship waits for a key.



Last life lost: Game Over, everything freezes.

README answers

QUESTION	ANSWER
What feature did the agent build?	DRA-23: lives, falls, game over and retry.
Why did it pick it?	It was the open week-1 goal, a locked rule required it, and its values were ready. The alternatives were blocked by missing values (section 4).
What did I change by hand?	No hand edits to the code. QA found one bug (a lopsided heart), and the UI agent fixed it. My changes were decisions: I built game over and retry in the same feature, answered the open values (blink speed, heart size, the Game Over box) and wrote them into the GDD, approved the ship, checked the browser start screen by hand, and accepted 5 minor review notes.